Rapid Reprovisioning with Virtual Switch Root

Landon Davidson[1], Madison Byrd[2], Eric Johnson[3] | Travis Cotton[4] (Mentor), Devon Bautista[4] (Mentor)
[1]University of Washington – Seattle, [2]University of South Carolina, [3]New Mexico Institute of Mining & Technology, [4]HPC-DES (LANL)

Background

WHY THE NEED FOR RAPID REPROVISIONING?

The increasing use of on-demand cloud services in scientific workloads has prompted the need for more flexibility in High-Performance Computing (HPC) systems. In this work, we explore and compare non-traditional boot techniques to traditional booting methods (**Fig. 1**).

TRADITIONAL BOOT OVERVIEW

1. **POST**: The system is powered on and tests hardware.
2. **Firmware**: The motherboard's firmware loads and continues the boot. This is where the kernel, bootloader, or PXE would be loaded and executed.
3. **Kernel**: The Linux kernel starts, which begins initializing drivers and loads the Initramfs (Initial RAM FileSystem).
4. **Initramfs**: A minimal, early-boot system that the kernel uses to mount a local root device or netboot an image.
5. **Switch Root**: Once the final root is ready, the Initramfs runs a switch_root function to enter the new root and reload services.

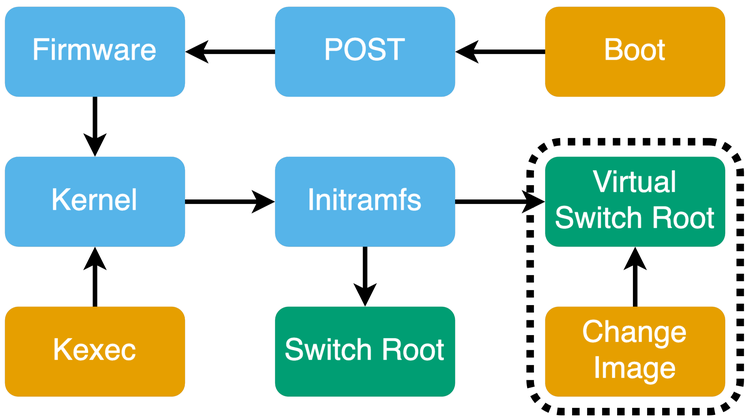

Figure 1. Flowchart of the traditional boot method, featuring our new Virtual Switch Root technique for rapid reprovisioning (highlighted in the dashed box).

Objectives

BRINGING FLEXIBILITY TO HPC

The primary focus was to replicate the flexibility of cloud computing environments by using a Virtual Switch Root technique. Specifically, we sought to utilize containers and virtual machines (VMs) to facilitate rapid and robust reprovisioning, enabling the ability to:

- **Transparently support** existing boot and OCI images
- **Swap node images instantly** by rebuilding the virtual switch root environment, bypassing standard hardware boot
- **Interface with an easy-to-use API** for rapid reprovisioning
- **Use a Unified Initramfs Image (UII)** that supports both container-based and VM-based switch root

All with minimal or even reduced resource overhead compared to traditional boot.

Technical Overview

WHAT IS VIRTUAL SWITCH ROOT?

Virtual Switch Root leverages a Unified Initramfs Image (UII) to enable rapid reprovisioning by seamlessly booting OCI container images (systemd-nspawn bootable containers) and existing boot images (QEMU VMs) through the following process:

1. **Build UII and specify boot parameters**
   Use provided script to build customized UII, as specified by provided Containerfile which gets distributed to compute nodes using a boot manager (such as OpenCHAMI).
2. **Pull the desired image into the Initramfs**
   Acquire the specified image using curl or Podman pull and prepare it for either QEMU or nspawn, respectively.
3. **Execute the Virtual Switch Root**
   When launching the image, the UII binds user-given devices, passes kernel boot parameters, and gives it full permissions on the host system.

Once booted, a user can quickly provision a new image on demand by repeating steps 2 and 3. This completely bypasses the physical reboot process and allows for rapid "reboots".

Performance Results

HOW DOES VIRTUAL SWITCH ROOT PERFORM?†

As a proof of concept, we conducted various benchmarks, highlighting key areas of performance improvement.

While our methods are not yet fully optimized for peak benchmark performance, they demonstrate that Virtual Switch Root can successfully support compute jobs.

QEMU VMs outperform traditional processing speed by **32.3%**, while nspawn underperforms by **8.2%**, according to HPL (**Fig. 2**).

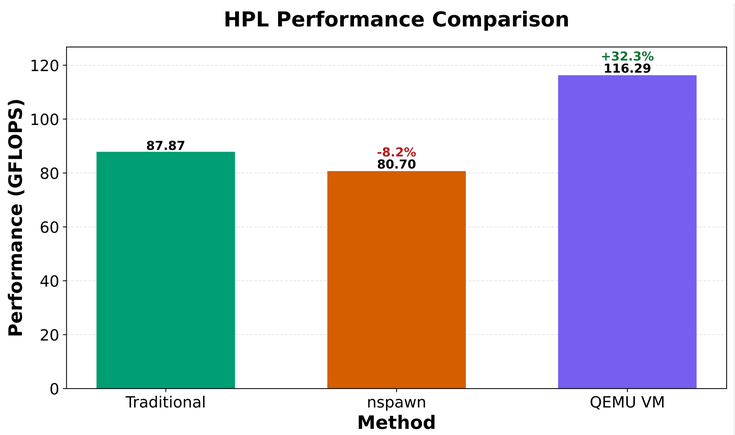

Figure 2. HPL Performance (GFLOPS) vs. Method. 3 nodes, 32 cores per node, 100 GB/node of RAM, 192 NB

†**NOTE:** Plotted data points represent the mean across three separate trials (n=3 nodes per trial). Node subset provisioning was rotated across trials to eliminate hardware bias and variance.

MPI AllReduce results (**Fig. 3**) indicate that network latency is minimal across all methods.

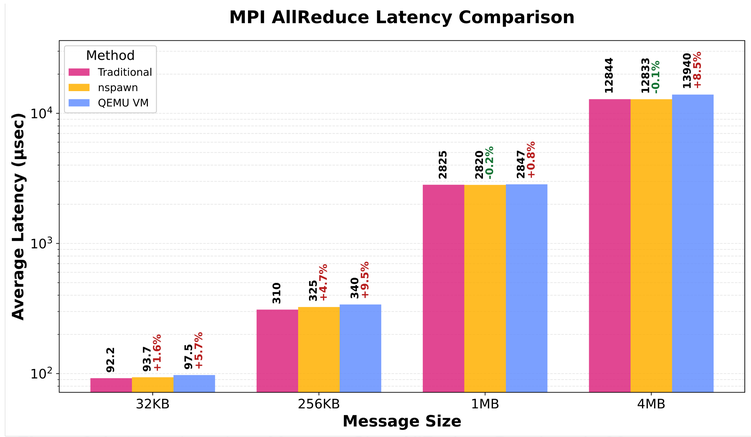

Figure 3. Latency (µsec) vs. Message Size. AllReduce Benchmark, 96 processes

STREAM memory results (**Fig. 4**) suggest that QEMU VMs significantly underperform by **~61%**, while nspawn is more promising with performance less than **2%** below traditional.

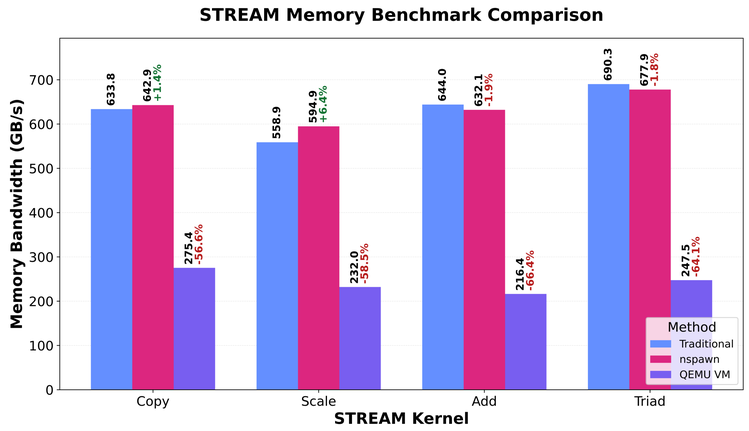

Figure 4. Memory Bandwidth (GB/s) vs. STREAM Kernel.

Conclusion

WHY USE VIRTUAL SWITCH ROOT?

Virtual Switch Root showcases that cloud-like flexibility is possible in HPC environments. By integrating with existing boot management infrastructure, our technique enables functional, quick reprovisioning with minimal setup.

FUTURE DIRECTION & IMPROVEMENTS

In the future we hope to see:

- An API for user-friendly remote reprovisioning
- Further investigation into benchmark results and optimization
- Balloon drivers to maximize resource allocation to VMs
- Optimized nspawn containers for full native performance
- Look into ways to increase memory performance with VMs

Los Alamos NATIONAL LABORATORY | NNSA National Nuclear Security Administration
Managed by Triad National Security, LLC, for the U.S. Department of Energy's NNSA.
LA-UR-26-26558
UNIVERSITY of WASHINGTON | UNIVERSITY OF South Carolina | NEW MEXICO TECH